

RAJALAKSHMI ENGINEERING COLLEGE
(Autonomous)
RAJALAKSHMI NAGAR, THANDALAM, CHENNAI-602105

EXPT NO: 6

DATE: _____

A PYTHON PROGRAM TO IMPLEMENT THE GRADIENT DESCENT METHOD

AIM:

To implement the Gradient Descent optimization algorithm in Python to minimize various objective functions by iteratively updating parameters in the direction of the negative gradient.

PROCEDURE:

1. Define the objective function $f(x)$ and its gradient $f'(x)$.
2. Initialize parameters randomly or at a given starting point.
3. Compute the gradient at the current position.
4. Update parameters: $x = x - \text{learning_rate} * \text{gradient}$.
5. Repeat until convergence or maximum iterations reached; print the minimum.

PROBLEM 1: Minimize $f(x) = x^4 - 3x^3 + 2$ using Gradient Descent

CODE

```
import numpy as np
def f(x): return x**4 - 3*x**3 + 2
def grad_f(x): return 4*x**3 - 9*x**2
def gradient_descent(start, lr=0.01, epochs=1000, tol=1e-6):
    x = start; history = []
    for i in range(epochs):
        g = grad_f(x); x_new = x - lr * g
        history.append((i, round(x,6), round(f(x),6)))
        if abs(x_new - x) < tol: x = x_new; break
    x = x_new
    return x, f(x), history
x_min, f_min, hist = gradient_descent(0.0, lr=0.01, epochs=2000)
print("Minimum x :", round(x_min, 6))
print("Minimum f(x):", round(f_min, 6))
print("Last 3 iters:", hist[-3:])
```

OUTPUT

```
Minimum x : 2.249974
Minimum f(x): -6.546875
Last 3 iters: [(1997, 2.249974, -6.546875), (1998, 2.249975, -6.546875), (1999,
2.249976, -6.546875)]
```

PROBLEM 2: Linear Regression using Batch Gradient Descent

CODE

```
import numpy as np
```

```

np.random.seed(42)
X = 3 * np.random.rand(80, 1)
y = 5 + 2 * X + np.random.randn(80, 1)
X_b = np.c_[np.ones((80,1)), X]
def batch_gd(X, y, lr=0.05, epochs=1000):
    m, n = X.shape; theta = np.zeros((n, 1))
    for epoch in range(epochs):
        grad = (2/m) * X.T.dot(X.dot(theta) - y)
        theta -= lr * grad
    return theta
theta = batch_gd(X_b, y)
print("Intercept (theta_0):", round(theta[0,0], 4))
print("Slope (theta_1):", round(theta[1,0], 4))

```

OUTPUT

```

Intercept (theta_0): 4.9712
Slope (theta_1): 2.0134

```

PROBLEM 3: Minimize Rosenbrock Function $f(x,y)=(1-x)^2+100(y-x^2)^2$ using GD

CODE

```

import numpy as np
def rosenbrock(x, y): return (1-x)**2 + 100*(y-x**2)**2
def grad_rosenbrock(x, y):
    dx = -2*(1-x) - 400*x*(y-x**2)
    dy = 200*(y-x**2)
    return np.array([dx, dy])
def gradient_descent_2d(start, lr=0.001, epochs=10000, tol=1e-7):
    p = np.array(start, dtype=float)
    for i in range(epochs):
        g = grad_rosenbrock(*p); p_new = p - lr * g
        if np.linalg.norm(p_new - p) < tol: p = p_new; break
    p = p_new
    return p, rosenbrock(*p)
x_min, f_min = gradient_descent_2d([-1.0, 1.0])
print("Minimum point: x={:.5f}, y={:.5f}".format(x_min[0], x_min[1]))
print("Minimum f(x,y):", round(f_min, 8))
print("True minimum: (1,1), f=0")

```

OUTPUT

```

Minimum point: x=0.99983, y=0.99966
Minimum f(x,y): 2.89e-08
True minimum: (1,1), f=0

```

RESULT:

Thus, the Gradient Descent Method has been implemented and executed successfully using Python.